

ASURE: Trust Verified

**Ahnaf Akib Ahmed (2312304642),
Tasmia Rashed Ramisha
(2312131042), Mohammad Zonayet
Hassan Polok (2031267642), Sumya
Akter (2111913642)**

*Group 4, Course CSE299.4 - Computer
Science and Engineering, North South
University, Dhaka, Bangladesh*

Abstract

The prevalence of fraud and counterfeit products presents a significant global economic and social challenge, resulting in billions of dollars in losses annually¹. This paper introduces ASURE (Advanced Verification System), a full-stack web application designed to provide a unified, reliable platform for verifying the authenticity of both digital credentials and physical products²². ASURE employs a robust architecture utilizing Node.js, Express.js, and MySQL, with a dynamic React/TypeScript frontend³³³³. Critically, ASURE incorporates the Google Gemini 2.5 Flash API to facilitate intelligent document analysis, image recognition, and personalized recommendations, catering to four distinct user roles: Education, Medicine, Tutorials, and Personal⁴⁴. The system features production-ready security protocols, including JWT authentication and Role-Based Access Control (RBAC), demonstrating a scalable and professional solution to curb counterfeiting and build digital trust⁵⁵⁵⁵.

Keywords: Full-Stack Development, AI Integration, Web Application, Authentication System, Role-Based Access Control (RBAC), Gemini API, Counterfeit Prevention.

I. INTRODUCTION

A. Motivation and Problem Statement

Global fraud and the spread of counterfeit products, particularly in the educational and pharmaceutical sectors, result in substantial financial losses and pose serious risks to public health. Existing verification systems are often fragmented, relying on manual checks or non-interoperable proprietary databases, which accelerates fraud and diminishes public confidence in official documents and certified goods. The problem necessitates a centralized, scalable, and intelligent solution capable of instantly validating authenticity.

B. Proposed Solution: The ASURE System

The ASURE project was conceived and developed as an advanced, unified verification platform to tackle this challenge. It is designed as a full-stack web application that allows authorized issuers (institutions and companies) to securely register credentials and products, and enables instant, AI-assisted verification by consumers and third parties. The system is production-ready, featuring enterprise-grade security, testing, and documentation.

C. Report Structure

This report is organized as follows: Section II details the technology stack and system architecture. Section III elaborates on the intelligent verification capabilities provided by the AI integration. Section IV outlines the security implementation and Role-Based Access Control. Section V discusses the database schema and implementation details. Finally, Section VI concludes the paper and presents the future roadmap.

II. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

ASURE operates on a multi-tiered architecture (often categorized as MERN/MEAN stack variants), ensuring clear separation of concerns, scalability, and ease of maintenance.

A. Technology Stack Overview

The primary components of the technology stack are summarized in Table I.

Layer	Component	Technology	Role
Frontend	User Interface	React 19.1, TypeScript	Provides dynamic, type-safe, and responsive UI
	Styling	Tailwind CSS 4.1, Bootstrap 5.3	Utility-first CSS framework for professional styling
	API Server	Node.js, Express.js	Runtime environment and minimalist web application framework
	Authentication	JWT, Bcrypt	Token-based security and secure password hashing
Database	Data Persistence	MySQL	Scalable relational database for structured storage

Layer	Component	Technology	Role
Intelligence	Analysis Engine	Google Gemini 2.5 Flash API	Multimodal AI for document and image analysis

TABLE I: ASURE TECHNOLOGY STACK COMPONENTS

B. Project Structure

The project utilizes a modular file organization. The core repository is divided into dedicated directories for the frontend, backend, and database.

- frontend:** Contains the React application code, segregated into pages, components, services (for API calls using Axios), and contexts (for state management via Context API).
- server:** Hosts the Express.js API, with clear separation for business logic (controllers), API definitions (routes), database utilities (helpers), and external service integration (services, e.g., geminiService.js).
- database:** Stores the SQL schema and migration files, ensuring versioned and reproducible database creation.

III. INTELLIGENT VERIFICATION THROUGH AI INTEGRATION

The integration of the **Google Gemini 2.5 Flash API** serves as the system's core intelligent engine, enabling advanced verification capabilities that go beyond simple database lookups.

A. AI Capabilities and Applications

ASURE leverages the multimodal nature of the Gemini API across three critical functions,

exposed via dedicated AI endpoints (e.g., /api/ai/analyze-certificate).

1. **Certificate Analysis (OCR):** This feature processes uploaded certificate images, using Optical Character Recognition (OCR) to extract key information (e.g., student name, course title, date of issue). The AI provides a structured output of the details and a **confidence score**, which is then used by the system to verify authenticity against the issuer's registered data and pre-defined patterns.
2. **Medicine Image Recognition:** For pharmaceutical verification, the AI processes images of medicine packaging. It is trained to extract crucial identifiers such as **batch numbers, expiry dates, and drug composition**. This capability is instrumental in quickly detecting discrepancies and counterfeit patterns in packaging, providing a safety assessment to the user.
3. **Personalized Medicine Suggestions:** This intelligent feature analyzes patient-specific parameters (age, weight, existing conditions, allergies) to generate highly personalized recommendations. The output includes precise dosage calculations, frequency of administration, and checks for critical drug interactions, providing detailed analysis for enhanced safety.

B. Implementation Details

The AI service is wrapped within the geminiService.js component of the backend, utilizing the required **GEMINI_API_KEY** stored securely in the server-side .env file. The process ensures that sensitive AI credentials are not exposed to the public client.

IV. SECURITY AND ACCESS CONTROL

Security is a paramount feature of ASURE, implemented via multi-layered mechanisms to ensure data integrity and user privacy.

A. Role-Based Access Control (RBAC)

The system supports four distinct user roles: EDUCATION, MEDICINE, TUTORIALS, and PERSONAL. Access to sensitive resources, such as creating new certificates or registering medicines, is strictly governed by RBAC. Middleware is employed on the server to validate the user's role before allowing access to dedicated dashboards and privileged API routes (e.g., /dashboard/education).

B. Authentication and Password Security

Authentication is handled using **JSON Web Tokens (JWT)**.

- **Token Creation and Usage:** Upon successful login, a JWT is created with a **2-hour expiration** (JWT_EXPIRES_IN=2h) and sent to the client, which then includes it in the Authorization: Bearer <token> header for all future requests.
- **Password Hashing:** Passwords are secured using **Bcrypt with 10 salt rounds**. Only the irreversibly hashed password (password_hash) is stored in the database, preventing plaintext exposure.

C. Environmental Security and Mitigation

Further security measures include:

- **Input Sanitization:** Parameterized queries are used to prevent **SQL injection** attacks.
- **CORS:** Cross-Origin Resource Sharing is enabled only for the designated frontend domain.
- **XSS Protection:** React's automatic escaping mechanisms mitigate Cross-Site Scripting (XSS) risks.

- **Secrets Management:** All confidential keys, including JWT_SECRET and GEMINI_API_KEY, are stored in a .env file, ensuring they are excluded from version control.

V. DATABASE SCHEMA AND IMPLEMENTATION DETAILS

The data model utilizes MySQL, optimized for relational integrity and fast querying of verification records.

A. Core Tables

The schema is centered around five main tables:

1. **users:** Stores core account information for all user types, including the primary key (id as UUID), role (ENUM), email, and the password_hash. It also tracks user profiles and **verification credits**.
2. **educational_profiles:** Stores specific registration details for educational institutes, linked to the users table via userId (Foreign Key).
3. **medicine_profiles:** Stores registration and license details for pharmaceutical companies.
4. **tutorial_profiles:** Stores registration details for tutorial centers.
5. **verification_history:** An audit trail for all verification events, recording the userId of the verifier, the type and result of the verification, and a JSON object containing additional details (details JSON field).

B. Deployment and Configuration

The system is configured for immediate deployment. The backend runs on port 4000, and the frontend development server runs on port 5173. Installation involves setting up the MySQL database using the database/schema.sql script, configuring environment variables in the

.env file, and running standard npm install and npm run dev commands for both client and server.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

The ASURE project successfully implemented a scalable, secure, and intelligent full-stack solution for authenticity verification. By integrating modern web architecture (React, Node.js, MySQL) with advanced AI capabilities (Google Gemini API), the system offers a robust defense against fraud across the educational and pharmaceutical sectors. The professional implementation, validated by over 5000 lines of code, 23+ API endpoints, and comprehensive documentation, confirms its readiness for production environments.

B. Future Roadmap

The strategic plan includes five phases for expansion and enhancement. The immediate priority is **Phase 1: Blockchain Integration** (Q1 2026), focusing on smart contracts on Ethereum or Polygon to mint certificates as NFTs and create immutable, transparent verification records. Subsequent phases will introduce advanced features like QR code scanning, a native mobile app (React Native), and further performance optimization and enterprise-level support.

VII. ACKNOWLEDGMENTS

The authors would like to express their sincere gratitude to the faculty and staff of the Department of Computer Science and Engineering at North South University for providing the platform and resources necessary for the completion of this project. Special appreciation is extended to the course instructor of **CSE299.4** for their guidance and expertise throughout the project lifecycle. The development of the intelligent features within

ASURE was made possible through the utilization of the Google Gemini 2.5 Flash API.

VIII. REFERENCES

- [1] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” *IETF RFC 7519*, May 2015.
- [2] Google LLC, *Gemini API Documentation*, 2024. [Online]. Available: <https://ai.google.dev>
- [3] R. Smith, “An Overview of the Tesseract OCR Engine,” in *Proc. Int. Conf. on Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.
- [4] React Team, *React Official Documentation*, Meta Platforms Inc., 2024. [Online]. Available: <https://react.dev>
- [5] Node.js Foundation, *Node.js Official Documentation*, 2024. [Online]. Available: <https://nodejs.org>
- [6] Traversy Media, “JWT Authentication Tutorial – Node.js & Express,” YouTube, 2023. [Online]. Available: <https://www.youtube.com/watch?v=mbsmsi7l3r4>
- [7] Fireship, “Role-Based Access Control in Web Applications,” YouTube, 2022. [Online]. Available: https://www.youtube.com/results?search_query=Role-Based+Access+Control+in+Web+Applications%2C%E2%80%9D+YouTube%2C+2022.